crash-safe ORAM

Final project in Advanced Topics in Online Privacy and Cybersecurity (67515)

Mohammad Awawda

Introduction:

ORAM is a way to store data on an untrusted server without letting the server learn which logical address the client is using. Encrypting the data is not enough by itself, because the server can still watch which locations are read and written. Path ORAM solves this by storing encrypted blocks in a binary tree. For every logical access, the client reads one complete path from the root to the leaf, moves blocks through a small local stash, assigns the accessed block to a fresh random leaf, and writes a complete path back

In this project, we extended the Path ORAM implementation to make it resilient to client crashes without losing data, while ensuring that the recovery process does not leak any information (including access patterns) to the server

Problem and motivation:

Path ORAM protects the access pattern of a client by storing encrypted blocks in a tree on the server. The client stores locally some of the sensitive information such as the position map, encryption key, and stash. Normally, during every access, the client reads one path, updates its local state, chooses a new random leaf for the block, and writes an updated encrypted path back to the server

The main problem we address is what happens if the client crashes during an access, a path ORAM access is made of several steps, and they dont happen atomically. The client changes local information while also sending changes to the server, and because of that, a crash can happen between any of these steps with zero guarantees to where it might occur.

For example, the client may update the position map but crash before finishing the path write update to the server, and the server may receive only part of the updated path. The process may also crash after writing recovery information but before marking the operation as completed. In a power failure, some data that was written to the disk may also disappear if it was not synchronized well.

This creates a problem because after restarting, the client has to decide what state is correct and how to safely continue. If the recovery isn't done correctly, blocks may be lost, duplicated, or the client and server may disagree about the location of a block.

Another important requirement is that recovery must not break the original privacy guarantees of ORAM. This matters because ORAM is usually used when both the data and the access pattern are sensitive, and we have at least a curious server which is monitoring the client’s access patterns. A simple recovery method could read many extra paths or search the server for a missing block, but these extra accesses would be

visible to the server and could reveal information about what happened before the crash. Protecting privacy during normal operation is not enough if one unexpected shutdown can force the client to recover in a way that leaks information. So a practical ORAM system should be able to recover reliably while still preserving the same privacy guarantees it provides during a normal execution

So our goal is to make Path ORAM recoverable while keeping its normal access pattern privacy. so that after a crash the system can return to a consistent state without losing data and without revealing additional information to the server.

Attacker Model and our goals:

We assume that the server is honest but curious for privacy. It runs the read and write API, but it is allowed to inspect everything it can see from the protocol like the leaf number, whether a request is a read or a write, the request size, and the timing of requests. It also stores the encrypted bucket contents and it should not learn which logical address the client is reading or writing.

We also consider that the client process may crash at any point, or a power failure may remove the unsynchronized end of the local log, or even that a second crash may happen during recovery. We don't assume that these faults are malicious, but the design must still recover to a valid state.

The client and its persistent state are trusted, this is an important assumption because the checkpoint stores the encryption key and position map, and the WAL (wal.py) stores recovery information that includes plaintext block data. If an attacker can read the client state directory, ORAM privacy is already lost. We also assume that one client controls the tree.

For integrity, using AES with GCM mode lets the client detect modified ciphertext blocks via verification tag. However, the project does not try to prevent DOS, a deletion of the whole server tree, or rollback attacks by a fully malicious server or a faulty one. A server can always stop answering. The TLS connection protects traffic from outside network observers, but the local self signed certificate used in the project is only for testing.

Our Goals:

- **Data confidentiality and integrity:** the server should not learn the plaintext blocks, and any change to a ciphertext is detected (with AES-GCM). moving, replaying or deleting whole blocks is not detected.
- **Access pattern privacy:** the server should not be able to learn which logical address is being accessed.
- **Crash consistency:** after a crash, the client should be able to recover to a valid state without corrupting the ORAM structure and data correctness.
- **Recovery privacy:** recovery must not leak the access pattern either. It only rewrites the path P(x) the server already saw, and the new leaf x' is never sent.
- **Repeatable recovery:** a crash during recovery is safe. Running recovery again gives the same result.
- **Durability of acknowledged accesses:** once an access returns, it survives any crash, because its PLAN is saved on disk (fsynced) before the path is written.
- **Fresh ciphertexts on every write:** every write re-encrypts with new nonces, including the redo. So the server can't tell which blocks changed, or whether a write is a redo.
- **Bounded recovery cost:** recovery makes at most one path write, and periodic checkpoints keep the log short.
- **Secure channel:** client and server talk over HTTPS, so a network attacker sees only what the server sees.
- **Reasonable efficiency:** an access should require only one tree path rather than downloading the whole database, which defeats the purpose of outsourcing.
- **Low overhead:** crash safety should cost little more than the plain Path ORAM. Measured at about 12-16% (in bench.py)
- **Verifiable:** an automated crash test injects crashes at every point and checks both the data and the server's trace. Runs are repeatable by the seed (explanation on that later).

Our design:

Main components:

Component	Role
client.py	Path ORAM logic, position map, stash, checkpoints, access protocol (read, plan, write, commit), crash injection and recovery.
crypto.py	AES-256-GCM encryption with fresh nonce every block. The address is inside the plaintext. Real blocks and dummies have the same length and structure (by encryption).
wal.py	the log of the client, writes PLAN and COMMIT records with length and CRC32 so a half written record at the end is found and cut
rpc.py	HTTPS client used to read/write paths and query server parameters and traces
workload.py	Generates the same random operations from a seed, and computes the final value each address should have
server.py	Stores the encrypted binary tree and records only what the server can observe
crashtest.py	Fault-injection tests, correctness/privacy checks, and performance measurements.
bench.py	Performance measurements, compares no log, log without fsync and full protocol over several tree heights
Tests directory	Unit tests for encryption, log encoding, eviction and checkpoints

Normal access protocol:

For every read or write, the client performs the following steps:

1. Look up the current leaf old leaf (which is position[address]) and independently choose a fresh random new leaf
2. Read the complete path from the root to old leaf. Every encrypted real block on the path is decrypted and placed in the client stash. Dummy blocks are ignored.
3. Perform the logical operation in the stash. Every access returns the value the block had before. a write then replaces the old value (and we pad it to the block size). Then update the position map so the address now belongs to new_leaf.
4. Run greedy eviction for old leaves. Starting from the deepest level, place up to Z stash blocks in each bucket. Blocks that cannot be placed stay in the stash.
5. Create a WAL PLAN record that contains the access sequence number, operation, address, old and new leaves, the return value, the real block that will be placed in each bucket of the path, and the remaining stash. Only the blocks containing a plaintext are saved, without dummies or ciphertexts, so the log stays small and a redo encrypts the path again with fresh nonces. In the normal crash safe mode, this PLAN is synced (with fsync) before the server is ever modified.
6. Encrypt every real block using AES in GCM mode with fresh nonce, fill every bucket to exactly Z slots with encrypted dummy blocks, and write the full old leaf path back to the server.
7. Append a small COMMIT record. It is not synced with the disk immediately. A process crash cannot lose it, since the record was already handed to the operating system (which lives inside the kernel), only a power cut or operating system crash can lose it. If it is lost, the recovery rewrites the same old leaf path from the PLAN with the same blocks encrypted again with fresh nonces, which is safe and avoids extra syncs on every access(using fsync).
8. Every K accesses, save a checkpoint of the whole client state which includes: the position map, the stash and the sequence number. it‘s written to a temp file, synced and then renamed over the old checkpoint. The rename is atomic, so after a crash we have either the old or the new checkpoint and never a mix of both. After that, the log is emptied so it does not grow forever, and the recovery only replays the records after that checkpoint

This ordering is the main rule of the design, the durable recovery plan is created before the remote state can be changed

Encryption and what the server sees:

Each slot is encrypted with AES-256 with GCM mode. The plaintext contains the logical block address and a fixed size data field, so real and dummy slots have the same ciphertext length. Every encryption uses a fresh 12-byte nonce and produces an authentication tag.In GCM the key and the nonce together decide the keystream, so a new nonce gives a different ciphertext and tag even for the same data. This way the server cannot compare ciphertexts to see which blocks stayed the same, or tell that a recovery write repeats old data. Reusing a nonce with the same key would also break GCM's security. Real and dummy slots have the same ciphertext length, and encrypting the same block twice produces different ciphertext, therefore any change to a ciphertext is detected. Dummy blocks use special addresses and random data.

The server stores only ciphertext buckets. For each logical access it sees a read of one leaf path and a write of the same leaf path, the leaf is random and has no relation to the logical address. It cannot tell if the access was a read or a write or which slots are real. It does not receive the position map, stash, encryption key, logical address, or the newly chosen leaf. After a crash it only sees one more write to a path it already saw. HTTPS is also used between the client and server so an outside network observer cannot directly read the JSON requests or ciphertext payloads.

Write ahead log and recovery

wal.py has two record types: PLAN and COMMIT. Records are constructed to be : [length][CRC32][payload]. On restart, the client reads valid records until the first incomplete or corrupt tail cuts anything after the last valid one. This is important because a power loss can stop in the middle of a file access.

Recovery starts from the last checkpoint. It replays every later PLAN into memory by restoring the saved new position and stash state. A matching COMMIT means that access is finished. If the last PLAN has no COMMIT in the log, it is treated as pending. The client then rewrites the saved real blocks of each bucket to pending.old leaf path is encrypted again with fresh nonces and new dummies, and appends a COMMIT which is synced with disk. Recovery never reads from the server

The table below shows, for each point where the client can crash, what is on disk afterwards, what recovery does, and what the server sees as a result:

Crash point	On disk after crash	Recovery action	Server sees
Before read	Last committed state	Nothing	Nothing extra
After read, before PLAN is durable	No PLAN (a torn one is cut by CRC)	Nothing; the access is dropped	A read of P(x) with no write
After PLAN, before or during the path write	PLAN, no COMMIT	Rewrite P(x) from the PLAN, then a synced COMMIT	One write of P(x)
After the write, COMMIT not written or lost in a power cut	PLAN, no COMMIT	Same redo	A second write of P(x)
After COMMIT	PLAN and COMMIT	Replay into memory only	Nothing
During checkpoint replacement	Old or new checkpoint, never a mix	Skip records already in the checkpoint	Nothing
During a recovery write	PLAN, no COMMIT	Redo again	Another write of P(x)

Writing the same blocks to the same path again gives the same final server state after decryption, only the ciphertexts are different. It also preserves the ORAM privacy goal during recovery. The server may notice an extra recovery write, but the leaf path does not contain new information because the old leaf path was already visible in the original path read, and the new leaf is never sent. The crash test explicitly checks that recovery makes no reads, at most one write and that its leaf belongs to the small set of paths already revealed before the crash

Privacy argument: the recovery sends only writes and only to a leaf that the server saw, so the server's view with recovery is the normal view plus some extra writes to known leaves at crash times, this can be done without knowing any extra logical addresses.

Limitations: if the client crashes after reading P(x) but before the PLAN is durable, the access is dropped and block stays mapped to x, if that address is accessed again later, the server sees P(x) read a second time, this links the two accesses which a regular ORAM never allows to happen. A fix is to sync a small INTENT record (address, x, x′) before the read, and finish the recovery on P(x), because the server has already seen that path, right before the crash. We did not implement this because it costs a second sync (about 1 ms) per access

Checkpoints:

If the WAL was kept forever, it would keep growing and every restart would have to replay all of it, so every K accesses (200 by default) the client writes a checkpoint. It holds the last sequence number, the tree parameters, the AES key, the position map and the stash.

Replacing the checkpoint is crash safe too. The client writes the whole checkpoint to a temporary file, syncs it, and then replaces the old one with an atomic rename (and syncs the directory, except on Windows it raised bugs). Only then is the WAL emptied. So a crash leaves either the old or the new complete checkpoint, never a mix, and if it happens before the WAL is emptied, recovery just skips the records already in the checkpoint.

Design analysis:

Security: The server cannot read block contents because all slots are encrypted, and AES-GCM also detects changes in a ciphertext. Fresh nonces and fixed size dummy slots prevent easy equality or real/dummy tests. Access pattern privacy follows the Path ORAM structure, the block is remapped to a random secret leaf after every access. During recovery, the design avoids opening a new path and only repeats an already observed one. There are two main limitations. First, GCM does not detect a malicious server that moves a valid ciphertext to another slot, rolls back to an older one, or erases a block. Second, the local information must stay trusted, because the WAL and checkpoint contain the key, the position map and plaintext blocks.

Efficiency: A normal access still uses one full path read and one full path write to the server. Crash safety adds local work: serializing a PLAN and, in the safe mode, one sync (with fsync) before the server writes. The PLAN stores only the real blocks of the path and the remaining stash, so it is small compared to the path (about 0.5 KB per access in our measurements). Checkpoints add O(N) local write for the position map. In our benchmark crash safety added about 0.4–0.8 ms per access +(12-16%). This is a deliberate tradeoff: the project spends local disk bandwidth and latency to get recovery. The unsynced COMMIT reduces part of this cost because losing a COMMIT only causes a safe redo.

Robustness to faults: The implementation includes crash points before the read, after the read, after the durable log, in the middle of the server write, after the write, after COMMIT, during checkpoint replacement, and even during a recovery write. A simulated power cut keeps only what was synced plus a random part of the rest, so it can also leave a half written record at the end of the log. If an unexpected error happens after the path is read, the client marks itself broken and refuses more accesses until it is reopened, which prevents continuing with memory that may be ahead of the server

Implementation and evaluation:

The project includes three types of evaluation. First, unit tests check the small building blocks: encryptions and tamper detection, fixed size dummy blocks, evictions, WAL framing and handling faulty suffixes (the work is durable and COMMITed until a certain point), and checkpoint corruption and atomic replacement. Second, crashtest.py runs generated workloads in child processes and intentionally kills the client at different crash points. It also tests power cut behavior and can crash a second time during recovery. After each run it checks that the stored data is valid, recovery touches no illegal path, and a clean second reopen has nothing left to redo.

Third, bench.py measures the cost of crash safety using the same random access workload in three modes. The nowal mode is the baseline Path ORAM with no recovery log. The nosync mode writes the log but does not fsync it, which mainly shows encryption/decryption and write cost. The sync mode is the real protocol and fsyncs every PLAN. The benchmark records average, p50 and p99 latency, operations per second, WAL bytes per operation, server bytes per operation, maximum stash size, and checkpoint count. It can sweep several tree heights and saves the results to results.csv. This comparison lets us separate the cost of the durability mechanism from the normal ORAM cost without changing the logical workload.

Mode	What is enabled	Why we measure it
nowal	No WAL	Baseline ORAM latency and bandwidth
nosync	WAL, no fsync	Cost of creating/writing log records
sync	WAL + fsync PLAN	Full crash-safe protocol cost

Benchmark results and analysis:

We ran `python bench.py --spawn-server --sweep` on one laptop, so the client and the server were on the same machine and used HTTPS on 127.0.0.1

Each configuration ran 3 times with 1000 timed accesses each, after 200 warm-up accesses that were not timed. We used $Z = 4$, 64 byte blocks and a checkpoint every 200 accesses. We tested heights 10, 12, 14 and 16 starting from an empty tree, and height 10 again with a full tree (every address written once before timing)

Average latency per access:

Height	Nowal (ms)	Nosync (ms)	Sync (ms)	sync overhead
10	6.83	6.96	7.78	14%
12	7.51	7.66	8.50	13%
14	8.07	8.26	9.00	12%
16	8.59	8.78	9.97	16%
10 (full)	6.98	7.18	7.97	14%

We can see that the nonsync is only 2-3% slower that nowal and that already includes writing the log and the checkpoints so the log itself is cheap so most of the cost comes from sync, on every PLAN it adds about 0.75-1.2ms per access, that's why we didn't sync the COMMIT, since it would add a second sync to every access.

The tree gets 64 times bigger from height 10 to 16, but the baseline is only about 25% slower, thats because the access go through only one path, so the sync does not depend on the height of the tree, so the overhead stays almost the same in all heights

The log adds about 520-560 bytes per access (935 with a full tree) which is relatively small compared to all the data sent to the server, since the server bytes and the max stash were the same in all three modes, so the log doesn't change anything about what the server sees

Since the server ran on a localhost, sending a request to the server took almost no time. With a remote server, every access would also wait for the requests to travel over the network and back, which usually takes much longer than 1 ms. The sync would still add the same 1 ms, but since each access would take longer overall, the overhead in percent would be smaller than what we measured

How to run and test the code:

The code is written in Python. It expects the package files client.py, crypto.py, rpc.py, wal.py and workload.py under a crashfreeORAM package directory

PyCryptodome is required for AES-GCM, and OpenSSL is used once to create the local TLS certificate

```
pip install pycryptodome
```

Create a test certificate (Windows Git Bash/PowerShell OpenSSL may need //CN=localhost instead of /CN=localhost):

```
mkdir certs
openssl req -x509 -newkey rsa:2048 -nodes -days 365 -keyout certs/key.pem -out certs/cert.pem -subj "/CN=localhost" -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

Run the unit tests:

```
python -m unittest discover -v
```

Run the server manually if desired:

```
python server.py -L 10 --slot-size 4
```

Run randomized crash testing. The --spawn-server option starts and stops the HTTPS server automatically:

```
python crashtest.py --spawn-server --runs 300 --ops 60 -L 10
```

Run the performance sweep. This compares nowal, nosync and sync and writes the measurements to results.csv:

```
python bench.py --spawn-server --sweep
```